

Transformer *Formulas* & Equations

A complete, annotated reference of every equation in the Transformer architecture — from input embeddings to the output projection — with explanations designed for practical implementation and research.



Uditya Narayan Tiwari

// Author & Maintainer

Attention Is All You Need

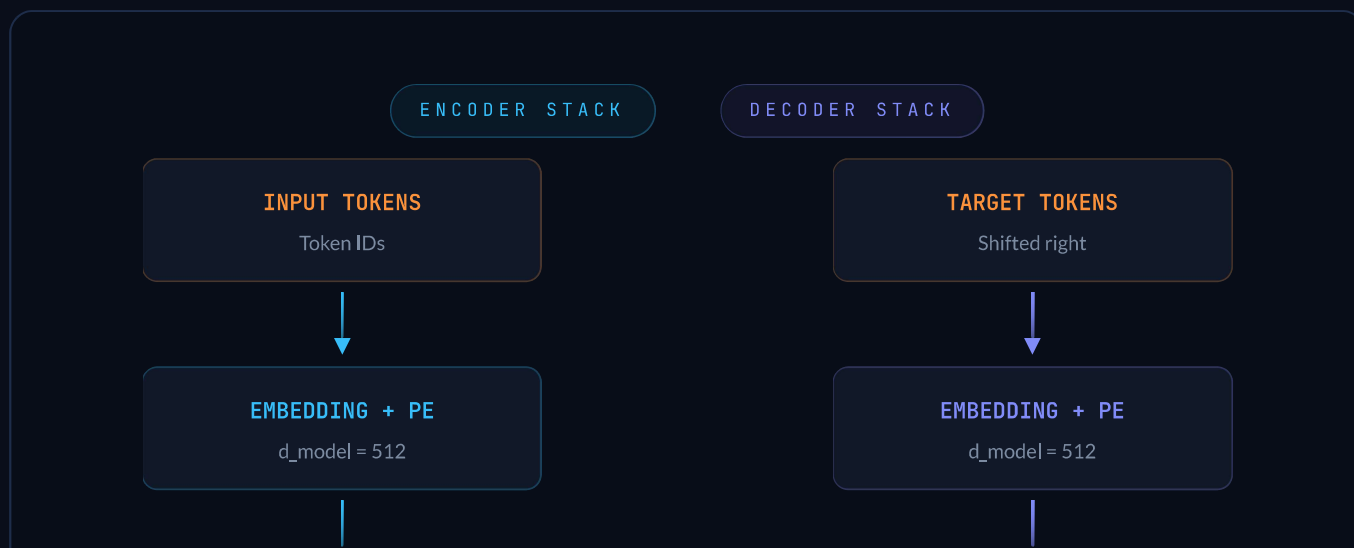
Encoder-Decoder

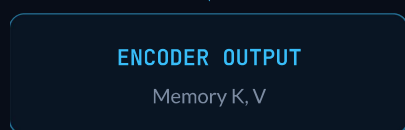
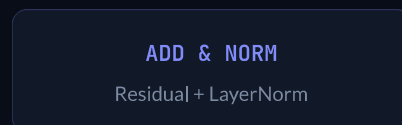
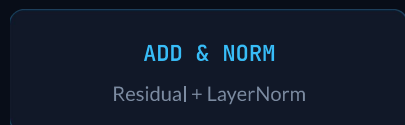
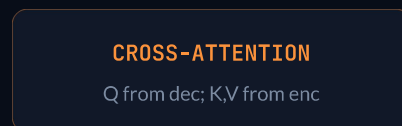
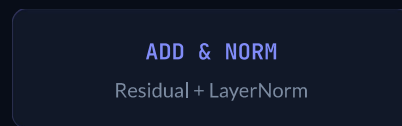
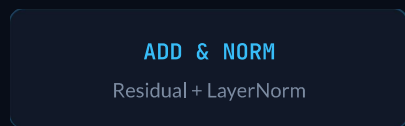
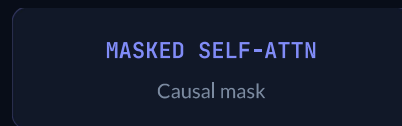
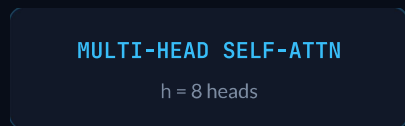
PyTorch / JAX Ready

01

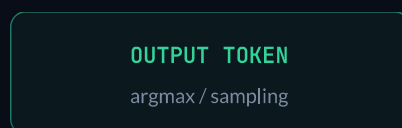
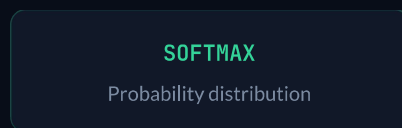
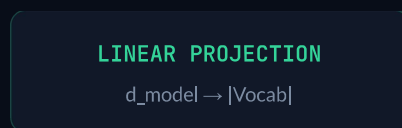
Architecture Overview

The Transformer (Vaswani et al., 2017) replaces recurrence entirely with self-attention, enabling massive parallelism. Both Encoder and Decoder are stacked blocks of the same sub-layers connected by residual paths.





K, V →



d_model = 512



h = 8 heads



N = 6 layers



d_ff = 2048

Hidden dimension

Parallel attention heads

Stacked blocks each side

FFN inner dimension



$$d_k = d_v = 64$$

Per-head key/value dim

02

Input Embeddings

Token IDs are mapped to dense vectors. The weight matrix is shared with the pre-softmax projection (weight tying), reducing parameters and improving generalisation.



Token Embedding

LEARNABLE

WEIGHT TYING

$$E = W_E \cdot x_{id} \cdot \sqrt{d_{model}}$$

FORMULA

where $W_E \in \mathbb{R}^{|V| \times d_{model}}$

Scaling by $\sqrt{d_{model}}$ keeps embedding magnitudes comparable to positional encodings, preventing positional signal from being drowned out.

 W_E Embedding matrix (vocab $\times$ d_{model}) x_{id}

One-hot token index

 $|V|$

Vocabulary size

 d_{model}

Model hidden dimension

Positional Encoding

Since Transformers have no recurrence, position is injected via a deterministic sinusoidal function of position and dimension index. It generalises to sequences longer than those seen in training.

Sinusoidal Positional Encoding

FIXED

SINUSOIDAL

FORMULA

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{2i/d_{\text{model}}})$$

$$\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{2i/d_{\text{model}}})$$

$$X_{\text{in}} = E + \text{PE}$$

Even dimensions use sine waves, odd dimensions use cosine waves with geometrically increasing wavelengths from 2π to $10000 \cdot 2\pi$. The model recovers relative positions via linear combinations of these signals.

pos Token position in sequence (0 ... T-1)

i Dimension pair index (0 ... $d_{\text{model}}/2 - 1$)

10000 Base frequency constant



Modern variants: BERT uses learned positional embeddings; GPT-NeoX and LLaMA use Rotary Positional Encoding (RoPE). Sinusoidal PE is parameter-free and still competitive for shorter contexts.

Scaled Dot-Product Attention

The core attention mechanism. Queries and Keys compute compatibility scores; Values are aggregated weighted by those scores. Scaling prevents gradient vanishing in high-dimension spaces.

FORMULA

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Step 1: Compute raw scores QK^T — each query against every key.

Step 2: Scale by $1/\sqrt{d_k}$ to keep variance ≈ 1 , avoiding softmax saturation.

Step 3: Softmax row-wise $\rightarrow$ attention weights summing to 1.

Step 4: Weighted sum of values V .

Q

Query matrix ($\text{seq_len} \times d_k$)

K

Key matrix ($\text{seq_len} \times d_k$)

V

Value matrix ($\text{seq_len} \times d_v$) d_k Key/Query dimension = d_{model} / h 

Why scaling matters: For $d_k = 64$, each dot product sums 64 products of values $\sim N(0,1)$, producing variance ≈ 64 . Without scaling, softmax saturates and gradients vanish.

05

Multi-Head Attention (MHA)

Instead of a single attention function, h parallel heads capture different relational aspects. Results are concatenated and projected back to d_{model} .



Multi-Head Formula

h HEADS

CONCAT + PROJECT

FORMULA

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

Each head has its own projection matrices $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$. Concatenated output ($h \cdot d_v = d_{\text{model}}$) is projected by $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

h Number of heads (8 in original paper)

$W_i^Q / W_i^K / W_i^V$ Per-head learned projections

W^O Output projection ($hd_v \rightarrow d_{\text{model}}$)

$d_k = d_v = d_{\text{model}} / h = 64$

06

Attention Masking

A padding mask ignores PAD tokens; a causal (look-ahead) mask in the decoder prevents attending to future positions.

Masked Attention Score

DECODER

CAUSAL

$$\text{score}_{\text{masked}} = \frac{QK^T}{\sqrt{d_k}} + M$$

FORMULA

$$\text{where } M_{ij} = \{ 0 \text{ if } i \geq j \mid -\infty \text{ if } i < j \}$$

Adding $-\infty$ causes softmax to output ≈ 0 for masked positions. In practice use $-1e9$ for numerical stability. The causal mask is a lower-triangular matrix of shape $(T \times T)$.



Causal mask shape: $M[i, j] = 0$ when $j \leq i$ (past visible), $-\infty$ when $j > i$ (future blocked). During training, this enables parallel computation over all timesteps.

07

Position-wise Feed-Forward Network

Each position's representation is independently passed through a two-layer MLP with ReLU. This introduces non-linearity and expands then contracts dimensionality.



FFN Formula

PER-POSITION

2-LAYER MLP

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2$$

FORMULA

$W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ expands to $d_{\text{ff}} = 2048$, ReLU is applied, $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ projects back. This expansion-contraction acts as a memory bank. Modern variants use **GeLU** or **SwiGLU** activations.

W_1, b_1 First linear transform + bias

W_2, b_2 Second linear transform + bias

d_{ff} Inner dimension ($\approx 4 \times d_{\text{model}}$)

$\max(0, \cdot)$ ReLU activation function

08

Layer Normalisation

Applied after each sub-layer to stabilise training. LayerNorm normalises across the feature dimension (d_{model}), unlike BatchNorm which normalises across the batch.



LayerNorm Formula

STABILISER

FEATURE-WISE

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

FORMULA

$$\mu = \frac{1}{d} \sum_j x_j \quad \sigma^2 = \frac{1}{d} \sum_j (x_j - \mu)^2$$

γ (gain) and β (bias) are learnable per-dimension affine parameters. $\epsilon \approx 1e-5$ prevents division by zero. **Pre-LN** (applied before sub-layer, as in GPT-2) is generally more training-stable than original post-LN.

μ, σ^2 Mean and variance over d_{model} features

γ, β Learnable scale and shift parameters

ϵ Numerical stability constant ($\approx 1e-5$)

$\odot$ Element-wise (Hadamard) product

09

Residual (Skip) Connection

Every sub-layer wraps its output in a residual addition, enabling gradients to flow directly to earlier layers and enabling training of very deep networks.

+ Add & Norm

GRADIENT FLOW

$$y = \text{LayerNorm}(x + \text{SubLayer}(x))$$

FORMULA

The SubLayer is either MHA or FFN. Adding the input x before normalisation ensures each layer learns residuals rather than full transformations — critical for training 6–96+ layers successfully.



Dropout: In the original paper, dropout ($p=0.1$) is applied inside $\text{SubLayer}(x)$ before the residual addition: $x + \text{Dropout}(\text{SubLayer}(x))$

10

Softmax Function

Converts raw logits to a probability distribution. Used in attention score normalisation and the final output projection.



Softmax

NORMALISATION

$$\text{softmax}(z)_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

FORMULA

For numerical stability, subtract the maximum before exponentiation: $\text{softmax}(z - \max(z))$. This is mathematically equivalent but avoids overflow.

11

Cross-Entropy Loss

The training objective — measures divergence between the model's predicted distribution and the true one-hot label over the vocabulary.



Cross-Entropy Loss

OBJECTIVE

TRAINING

$$\mathcal{L} = - \frac{1}{T} \sum_{t=1}^T \log p_{\theta}(y_t \mid y_{<t}, x)$$

FORMULA

Summed over all T target positions. Teacher forcing is used during training — ground-truth previous tokens are fed, not model predictions. This maximises the log-probability of the correct next token.

T

Target sequence length

y_t

Ground-truth token at position t

p_θ

Model probability (softmax output)

x

Source input sequence

12

Learning Rate Schedule (Warmup)

Custom LR schedule with linear warmup followed by inverse square root decay — avoids large, destabilising updates early in training.



LR Warmup Schedule

OPTIMISER

ADAM

$$\text{lr} = d_{\text{model}}^{-0.5} \cdot \min(\text{step}^{-0.5}, \text{step} \cdot \text{warmup}_{\text{steps}}^{-1.5})$$

FORMULA

For the first warmup_steps (≈ 4000) LR increases linearly. After that it decreases as inverse square root of the step. Adam optimiser with $\beta_1=0.9$, $\beta_2=0.98$, $\epsilon=1e-9$ is used throughout.

step

Current training step number

warmup_steps Steps for linear warmup (≈ 4000)**d_{model}**

Model dimension (normalises scale)

13

Label Smoothing

A regularisation technique that replaces hard one-hot targets with a soft distribution, preventing overconfident predictions and improving calibration.

☰ Smoothed Target Distribution

REGULARISATION

$$y_{\text{smooth},k} = (1 - \epsilon_{\text{ls}}) \delta_{ky} + \frac{\epsilon_{\text{ls}}}{|V|}$$

FORMULA

With $\epsilon_{\text{ls}} = 0.1$, the true class gets probability $0.9 + 0.1/|V|$ and all other classes share the remaining 0.1. This slightly hurts perplexity but improves translation BLEU score.

14

Rotary Positional Encoding (RoPE)

Used in LLaMA, GPT-NeoX, PaLM, and most modern LLMs. Encodes absolute position as rotation of Q/K vectors so dot products depend only on *relative* positions.

🔄 RoPE — 2D Rotation per Pair

RELATIVE POS

MODERN LLMs

$$q_{\text{rot}}(m) = R_{\theta,m} q$$

$$R_{\theta,m} = \text{diag}(R_2(m\theta_1), \dots, R_2(m\theta_{d/2}))$$

$$\text{where } \theta_i = 10000^{-2(i-1)/d} \quad R_2(\alpha) = [\cos\alpha, -\sin\alpha; \sin\alpha, \cos\alpha]$$

FORMULA

Each consecutive dimension pair in Q and K is rotated by angle proportional to token position m and frequency θ_i . The inner product $q_{\text{rot}}(m)^T k_{\text{rot}}(n)$ depends only on relative offset (m-n).



Why RoPE is preferred: Extrapolates better to unseen sequence lengths, integrates with Flash Attention. YaRN modifies the base frequency θ for context length extension.

15

Grouped-Query Attention (GQA)

Shares Key/Value heads across groups of Query heads, reducing KV-cache memory during inference without significantly degrading quality.

GQA Grouping

EFFICIENCY

LLaMA / Gemini

$$G = \frac{h_Q}{h_{KV}}$$

$$\text{GQA}(Q_i) = \text{Attention}(Q_i, K_{\lfloor i/G \rfloor}, V_{\lfloor i/G \rfloor})$$

FORMULA

KV-cache reduced by factor G . MHA is $G=1$; MQA is $G=h$ (single KV head). GQA (Ainslie et al., 2023) balances quality and efficiency; LLaMA 2/3 uses $G=4-8$.

h_Q Number of query heads

h_{KV} Number of key/value heads

G Group size = h_Q / h_{KV}

16

Complexity Analysis

Understanding computational and memory costs guides architectural choices for long-context models and resource-constrained deployments.

Time & Memory Complexity

BIG-O

BOTTLENECK

FORMULA

Self-Attention: $O(n^2 \cdot d)$ Memory: $O(n^2)$

FFN per layer: $O(n \cdot d^2)$

Total params $\approx 12 \cdot N \cdot d^2$

The **quadratic cost in n** is the primary bottleneck. Doubling the sequence quadruples attention compute. Flash Attention, Sliding Window Attention, and Linear Attention aim to reduce this. Factor 12 covers 4 MHA matrices (Q,K,V,O) + 2 FFN matrices per layer.



KV-Cache at inference: Cache size = $2 \times N \times T \times d_{\text{model}} \times \text{sizeof}(\text{dtype})$. For 70B models with long context this is the dominant memory cost.

Authored by **Uditya Narayan Tiwari** · Transformer Formulas Reference v1.0 · Based on Vaswani et al. "Attention Is All You Need" (2017) and subsequent work.

// INTERNAL DEVELOPER REFERENCE